

Анализ свойств строковых образцов, содержащих повторные переменные

Студент: Т. Т. Сатыбалдиев, ИУ9-82Б

Руководитель: А. Н. Непейвода

МГТУ им. Н.Э. Баумана

Кафедра «Теоретическая информатика и компьютерные технологии»

2026 г.

Уравнения в словах возникают в статическом анализе программ, верификации протоколов безопасности и символьном исполнении кода. Их решают **SMT-решателями** с теорией строк, например **Z3** и **Ostrich**.

Цель работы

Анализ однозначности строковых шаблонов с повторяющимися переменными.

1. Разработать **обёртку-анализатор** с базой уже разобранных шаблонов.
2. Разработать **эвристики** быстрой оценки однозначности и неоднозначности шаблона *вне решателя*.
3. Исправить и **ускорить отсечение ветвей** при переходе к уравнениям в словах в решателе Ostrich.

Шаблон $\mathcal{P}$ — слово над $\Sigma \cup X$: литералы (константы) и переменные.

Шаблон **неоднозначен**, если одна строка соответствует ему двумя различными способами:

$$\exists \sigma_1, \sigma_2 : X \rightarrow \Sigma^*, \quad \sigma_1 \neq \sigma_2, \quad \hat{\sigma}_1(\mathcal{P}) = \hat{\sigma}_2(\mathcal{P}) = w.$$

В противном случае шаблон **однозначен**.

Сведение к уравнению в словах

Задача определения однозначности сводится к выполнимости уравнения

$$\mathcal{P}[\sigma_1] = \mathcal{P}[\sigma_2] \wedge \sigma_1 \neq \sigma_2,$$

в котором каждая переменная встречается **ровно дважды** — по одному разу с каждой стороны. Например, для шаблона $x y x$:

$$x_1 y_1 x_1 = x_2 y_2 x_2.$$

Выполнимость уравнения в словах над конечным алфавитом **разрешима** (Маканин, 1977) и PSPACE-полна (Plandowski, 2004).

Последовательные элементарные преобразования сокращают длину решения (само уравнение может удлиняться). Для первых символов $u = v$:

- совпадают $\Rightarrow$ сокращаются;
- разные литералы $\Rightarrow$ ветвь отбрасывается;
- литерал против переменной $\Rightarrow x = \varepsilon$ **или** $x = a x'$.

Случай «**переменная против переменной**» порождает ветвление:

$$x = y \cdot r \quad \text{или} \quad y = x \cdot r, \quad r \text{ — свежая переменная.}$$

Случай «переменная против переменной»

Обе ветви *вводят новую переменную*, не уменьшая их общего числа $\Rightarrow$ возможно бесконечное дерево вывода.

Пример: структура воспроизводит себя

$$x x R_1 = y x R_2$$

Ветвь $x = y x'$ после подстановки:

$$y x' y x' R_1 = y y x' R_2 \xrightarrow{\text{сокращаем } y} x' y x' R_1 = y x' R_2.$$

Снова ведущие x' и y — та же структура $\Rightarrow$ ветвь не завершается.

Доработка решателя: классы коммутации (1/2)

Теорема (Fine–Wilf)

Если слово w имеет периоды p, q и $|w| \geq p + q - \gcd(p, q)$, то w имеет период $\gcd(p, q)$.

Следствие — критерий коммутативности (Lyndon–Schützenberger)

$uv = vu \iff u = z^k, v = z^m$ для общего корня z .

Лемма о коммутировании при удвоении

Если σ — решение уравнения $x^J y \Phi_1 = y^I x \Phi_2$ ($I, J \geq 1$), то $\sigma(x) \sigma(y) = \sigma(y) \sigma(x)$ — значения коммутируют (степени общего корня).

$\Rightarrow$ потенциально бесконечная ветвь заменяется **одним детерминированным упрощением**.

Доработка решателя: классы коммутации (2/2)

Класс коммутации — множество переменных, для которых уже доказано попарное коммутирование. Использование на шаге разбиения:

Нормализация без ветвления

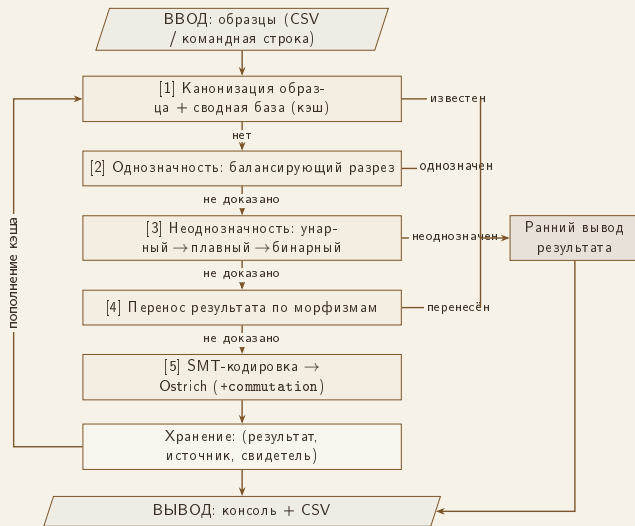
Если обе ведущие переменные в одном классе — общий блок переменных класса сокращается с обоих концов *без новых ветвей*.

Коммутационное разбиение / прямое разрешение длин

Если коммутирование ещё не установлено, но обнаружена структура леммы об удвоении (или порождающая её) — применяется специализированное правило.

Реализовано в Ostrich (Scala, фреймворк Princess) через `Plugin.AxiomSplit / AddAxiom`; управляется флагом сборки, публичный интерфейс теории строк не меняется.

Каркас системы



Каскад методов возрастающей вычислительной стоимости и поток данных.

Каскад эвристик вне решателя

Перед решателем работает каскад дешёвых структурных эвристик; методы упорядочены по возрастанию стоимости, решатель запускается последним.

Опорное понятие — балансирующая позиция: разрез $\mathcal{P} = w_1 w_2$, у которого векторы кратностей переменных пропорциональны, $\nu(w_1) = c \nu(w_2)$, $c \in \mathbb{Q}_{>0}$.

Критерий однозначности

Если $w_1 = x_i C_1 w'_1$ и $w_2 = x_i C_2 w'_2$ при $C_1 \neq C_2$ (одна переменная, разные константы), то образец однозначен по x_i (при двух переменных — в целом).

Критерий неоднозначности (каскад)

1. унарный — свидетели над одной буквой;
2. плавные границы — факторы периодического слова w^∞ ;
3. бинарный конструктивный перебор.

Строгость и перенос результата

Каждый свидетель проверяется обратной подстановкой $\Rightarrow$ **ложных срабатываний нет**. Результат переносится на родственные образцы по морфизмам.

Каждое решение нильсеновского разбиения (ветвление, нормализация, разложение) фиксируется логгером `DecisionTreeLogger` как узел дерева. Три представления:

- цветной вывод в терминал;
- самодостаточный интерактивный **HTML** с раскрытием узлов и легендой правил (коммутационное разбиение, нормализация по классам, нильсеновское разбиение — разными цветами);
- граф потока вывода **Graphviz** (SVG/PNG) при наличии `dot`.

доказательство unsat для образца xxxAyxVyx

Пример:

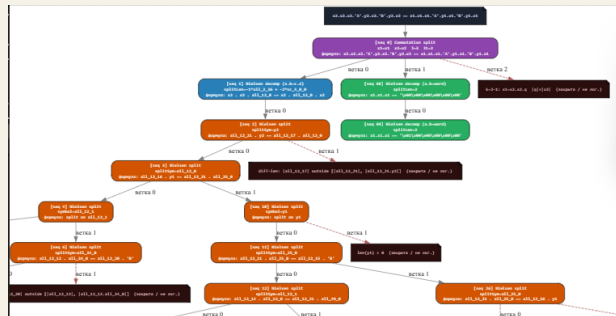
```
OSTRICH Nielsen-splitter derivation flow
90 decisions - source: decision_tree.json - read top--bottom: each box transforms the equation above it

x2,x2,x2,"A"x2,x2,"B"x2,x2 == x1,x1,x1,"A"x1,x1,"B"x1,x1

[1] Commutation split (same-graph)
NielsenCommutatSplit
x1=x2 x2=x2 >=3 31x2
условия:
x2,x2,x2,"A"x2,x2,"B"x2,x2 == x1,x1,x1,"A"x1,x1,"B"x1,x1
акции (taken/closed):
- comm: x2op,q, p,qop,p x1=(x2).p len(x2)<2*len(x2) / same
- k=3: x1=x2,x2,x2,q / closed
- k=3-1: x1=x2,x2,q [q]<[x2] / closed
- k=3 decision: Nielsen (3)

[2] Nielsen decomp (a,b=c,d) (Nielsen decomposition (a,b=c,d, [q]<[d])
NielsenDecomp
splitLen=1*x11_2_36 + -2*x1_8_8
условия:
x2,x2,x2,x11_2_8 == x2,x11_2_8,x2
вычисления (2):
- x2 = x2
- _8 = _8
- k=3 decision: Nielsen (3)
```

Интерактивный HTML-вид узлов



Граф потока вывода (Graphviz)

Эвристики (вне решателя)

На наборе из **118 шаблонов** мгновенно разрешают **77 %** задач, без единого ложного срабатывания; медиана < 1 мс.

Решатель Ostrich vs. Z3

- в среднем в **2,7** раза быстрее Z3 (на общих задачах);
- на *xAxByCyx* — до **1200** раз: 151 с $\rightarrow$ 0,12 с;
- классы коммутации разрешили **2** ранее зависавшие задачи.

Компоненты взаимодополняемы.

Ограничения

Выявлены непересекающиеся «слепые зоны» обоих алгоритмов — это обосновывает применение специализированных эвристик лишь для узких классов задач. Классы коммутации *не устраняют* бесконечное ветвление, а лишь упрощают прохождение части ветвей: даже на коммутативных классах возможны бесконечные циклы.

Выводы

Существенно расширена база однозначных и нетривиально неоднозначных образцов. Дальнейшее направление — адаптация эвристик (рассчитанных на высокую кратность переменных и специфические константы) к **большему числу переменных** и константам в виде **перекрывающихся слов**.

Спасибо за внимание!

Готов ответить на вопросы.